Занятие 5: Устройство backend-приложения

Архитектура, разделение ответственности, Dependency Injection

Слайд 1: Путь запроса внутри приложения

```
HTTP Request
    ↓
Router (маршрутизация)
    ↓
Handler (обработчик)
    ↓
Business Logic (бизнес-логика)
    ↓
Data Access (доступ к данным)
    ↓
Database
    ↓
Response
```

Каждый слой имеет свою ответственность

Слайд 2: Разделение ответственности компонентов

Router (Маршрутизатор)

- Определяет, какой обработчик вызывать
- Извлекает параметры из URL
- Не содержит бизнес-логики

Handler (Обработчик)

- Валидация входных данных
- Вызов бизнес-логики
- Формирование HTTP-ответа
- Не содержит бизнес-правил

Business Logic (Бизнес-логика)

- Реализация бизнес-правил
- Оркестрация операций
- Не зависит от HTTP

Data Access (Доступ к данным)

- Работа с БД
- SQL-запросы
- Не содержит бизнес-логики

Слайд 3: Зачем отделять бизнес-логику от HTTP и БД

Отделение от HTTP

- Логику можно использовать без HTTP (CLI, тесты)
- Легче тестировать
- Можно сменить фреймворк

Отделение от БД

- Логику можно использовать с другим хранилищем
- Легче тестировать (mock БД)
- Можно оптимизировать запросы независимо

Пример:

```
# Плохо: всё в одном месте
def handler(request):
    user = db.query("SELECT * FROM users WHERE id = ?", request.id)
    if user.age < 18:
        return error("Too young")
    return user

# Хорошо: разделение
def handler(request):
    user = user_service.get_user(request.id)
    return user

def get_user(user_id):
    user = db.get_user(user_id)
    if user.age < 18:
        raise ValidationError("Too young")
```

Слайд 4: Зависимости и Dependency Injection

Зависимость - компонент, который нужен другому компоненту

Примеры:

- Handler зависит от UserService
- UserService зависит от UserRepository
- UserRepository зависит от Database

Dependency Injection (DI)

- Передача зависимостей извне
- Не создание зависимостей внутри компонента

Плохо (создание внутри):

```
class UserService:
    def __init__(self):
        self.db = Database()  # жёсткая зависимость
```

Хорошо (внедрение):

```
class UserService:
    def __init__(self, db: Database):
        self.db = db  # зависимость внедряется
```

Слайд 5: Схема Dependency Injection

```
Application
    ↓ создаёт
UserService ←── UserRepository ←── Database
    ↑ внедряет
Handler
```

Преимущества DI:

- Легче тестировать (можно подменить зависимости)
- Гибкость (можно сменить реализацию)
- Явные зависимости

Слайд 6: Конфигурация отдельно от кода

Переменные окружения

```python
import os

DB_HOST = os.getenv('DB_HOST', 'localhost')
DB_PORT = os.getenv('DB_PORT', '5432')
API_KEY = os.getenv('API_KEY')
```

Файл конфигурации

```yaml
database:
  host: localhost
  port: 5432
  name: mydb
api:
  key: secret123
```

Почему важно:

- Разные конфигурации для разных окружений
- Секреты не в коде
- Легче менять без пересборки

Слайд 7: Побочные эффекты

Побочный эффект - изменение состояния вне функции

Примеры побочных эффектов:

- Запись в БД
- Отправка email
- Вызов внешнего API
- Запись в файл
- Изменение глобальной переменной

Чистые функции - без побочных эффектов

```
# Чистая функция
def add(a, b):
    return a + b

# С побочным эффектом
def save_user(user):
    db.insert(user)  # побочный эффект
```

Почему важно:

- Чистые функции легче тестировать
- Чистые функции предсказуемы
- Побочные эффекты нужно изолировать

Слайд 8: Назначение фреймворка

Фреймворк предоставляет:

- Маршрутизацию
- Обработку HTTP
- Middleware
- Интеграцию с сервером

Фреймворк НЕ определяет:

- Архитектуру приложения
- Бизнес-логику
- Структуру кода

Примеры фреймворков:

- Django (Python)
- FastAPI (Python)
- Express (Node.js)
- Spring (Java)

Выбор фреймворка - это инструмент, не архитектура

Слайд 9: Пример архитектуры

```
app/
├── handlers/          # HTTP обработчики
│   ├── user_handler.py
│   └── post_handler.py
├── services/          # Бизнес-логика
│   ├── user_service.py
│   └── post_service.py
├── repositories/      # Доступ к данным
│   ├── user_repository.py
│   └── post_repository.py
├── models/            # Модели данных
│   ├── user.py
│   └── post.py
└── config/            # Конфигурация
    └── settings.py
```

Слайд 10: Пример кода - Handler

```
def create_user_handler(request):
    # Валидация
    data = validate_user_data(request.json)

    # Вызов бизнес-логики
    user = user_service.create_user(data)

    # Формирование ответа
    return {
        "id": user.id,
        "name": user.name,
        "email": user.email
    }, 201
```

Ответственность:

- HTTP-специфичные вещи
- Валидация формата
- Формирование ответа

Слайд 11: Пример кода - Service

```
class UserService:
    def __init__(self, user_repo, email_service):
        self.user_repo = user_repo
        self.email_service = email_service

    def create_user(self, data):
        # Бизнес-правила
        if self.user_repo.email_exists(data.email):
            raise BusinessRuleError("Email exists")

        # Создание пользователя
        user = User(**data)
        user = self.user_repo.save(user)

        # Отправка email (побочный эффект)
        self.email_service.send_welcome(user.email)

        return user
```

Ответственность:

- Бизнес-логика
- Оркестрация
- Не зависит от HTTP

Слайд 12: Пример кода - Repository

```
class UserRepository:
    def __init__(self, db):
        self.db = db

    def save(self, user):
        return self.db.insert("users", user.to_dict())

    def find_by_id(self, user_id):
        data = self.db.query("SELECT * FROM users WHERE id = ?", user_id)
        return User.from_dict(data)

    def email_exists(self, email):
        count = self.db.query(
            "SELECT COUNT(*) FROM users WHERE email = ?",
            email
        )
        return count > 0
```

Ответственность:

- Работа с БД
- SQL-запросы
- Не содержит бизнес-логики

Слайд 13: Практическое задание

Задание 1: Разделение ответственности
Дан обработчик, который делает всё. Разделите его на слои.

Задание 2: Dependency Injection
Переделайте код с созданием зависимостей на DI.

Задание 3: Конфигурация
Вынесите конфигурацию в переменные окружения.

Слайд 14: Вопрос для собеседования

Вопрос: Разделите обязанности обработчика, который одновременно проверяет запрос, сохраняет данные и отправляет письмо.

Дано:

```
def handler(request):
    data = request.json

    # Валидация
    if not data.get('email'):
        return error("Email required")

    # Сохранение
    db.insert("users", data)

    # Отправка письма
    send_email(data['email'], "Welcome")

    return success(data)
```

Ожидаемый ответ:

1. Handler: валидация формата, вызов service, формирование ответа
2. Service: валидация бизнес-правил, вызов repository, вызов email service
3. Repository: сохранение в БД
4. EmailService: отправка email